

Reviewer Pack — Matroid Rank Gap in Monotone DNF for k-CLIQUE

Entry 8c66f5bea2d7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 17:16:43 UTC

Matroid Rank Gap in Monotone DNF for k-CLIQUE

Entry ID: 8c66f5bea2d7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 17:16:43 UTC

1. Conjecture

Field A (mathematical branch): Matroid Theory **Field B** (complexity object): Monotone DNF Formulas

Statement:

For any k-CLIQUE instance represented as a 3-CNF formula, the maximum rank of the matroid constructed from its clauses is $\Omega(n)$, whereas for random 3-CNF formulas of the same size, the maximum rank is $O(\log n)$.

Rationale (proposer's reasoning):

The structure of k-CLIQUE instances imposes a matroid with high rank, reflecting the inherent complexity of the problem, which necessitates larger monotone DNF formulas. Random instances, lacking such structured dependencies, have lower rank and thus can be expressed with smaller formulas.

Taxonomy category: BOUNDED_ARITHMETIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 403ead459da0536a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    rank = 0
    for j in range(cols):
        i_max = rank
        for i in range(rank, rows):
            if abs(matrix[i][j]) > abs(matrix[i_max][j]):
                i_max = i
        if abs(matrix[i_max][j]) < 1e-9:
            continue
        matrix[rank], matrix[i_max] = matrix[i_max], matrix[rank]
        for i in range(rows):
            if i != rank:
                factor = -matrix[i][j] / matrix[rank][j]
                for k in range(cols):
                    matrix[i][k] += factor * matrix[rank][k]
        rank += 1
    return rank

def generate_random_3cnf(n, m):
    clauses = []
    variables = list(range(1, n + 1))
    for _ in range(m):
        clause = random.sample(variables, 3)
        sign = [random.choice([-1, 1]) for _ in range(3)]
        clause = [(sign[i] * var) % n for i, var in enumerate(clause)]
        clauses.append(clause)
    return clauses

def generate_kclique_3cnf(n, k):
    if k > n:
        raise ValueError("k must be less than or equal to n")
    variables = list(range(1, n + 1))
    edges = [(i, j) for i in range(1, n) for j in range(i + 1, n + 1)]
    random.shuffle(edges)
    selected_edges = edges[:k * (k - 1) // 2]
    clauses = []
    for u, v in selected_edges:
```

```

        for w in variables:
            if w != u and w != v:
                clause = [u, v, w]
                sign = [random.choice([-1, 1]) for _ in range(3)]
                clause = [(sign[i] * var) % n for i, var in enumerate(clause)]
                clauses.append(clause)
    return clauses

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        kclique_clauses = generate_kclique_3cnf(n, int(0.5 * n))
        random_clauses = generate_random_3cnf(n, int(0.5 * n))

        m_kclique = gaussian_elimination([[1 if var in clause else 0 for var in range(1, n + 1)] for clause in kclique_clauses])
        m_random = gaussian_elimination([[1 if var in clause else 0 for var in range(1, n + 1)] for clause in random_clauses])

        results.append({
            "n": n,
            "m_kclique": m_kclique,
            "m_random": m_random
        })

    total_kclique = sum(result["m_kclique"] for result in results)
    total_random = sum(result["m_random"] for result in results)
    instances_tested = len(results) * len(n_values)

    conjecture_holds = all(m_kclique >= n for result in results for m_kclique in [result["m_kclique"] for result in results]) and \
        all(m_random <= math.log(n) for result in results for m_random in [result["m_random"] for result in results])

    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "matroid_rank_gap",
        "metric_value": total_kclique / instances_tested,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] * result["instances_tested"] for result in results) /
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 * result["instances_tested"] for result in results) /
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

```

```

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7d6ea076.py", line 105, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7d6ea076.py", line 74, in run_trial
    m_random = gaussian_elimination([[1 if var in clause else 0 for var in range(1, n + 1)] for clause in clauses])
               ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7d6ea076.py", line 26, in gaussian_elimination
    if abs(matrix[i_max][j]) < 1e-9:
        ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before collecting data; cannot verify support fraction or counterexamples |
next: Fix the gaussian_elimination index error and re-run tests with multiple seeds

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	118974
2	preregistration	ollama_remote	qwen3:8b	0	0	32065
3	novelty	ollama_remote	qwen3:8b	0	0	27475
4	novelty	ollama_remote	qwen3:8b	0	0	19321
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16080
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9613

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10447
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14094
9	judge	ollama_remote	qwen3:8b	0	0	19300

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 267369 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/8c66f5bea2d7.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/8c66f5bea2d7.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/8c66f5bea2d7.tar.gz (if generated)